

<< IERG 3810 >>

<< Microprocessor System Design Laboratory >>

Report on Experiment <<No.1>>

<< General Purpose Inputs/Outputs (GPIO) >>

Group: BL 01

Members: Yeung Man, Thomas (1155203181)

Wong Man Hei, Jimmy (1155213238)

Submission Date: 28/09/2025

Disclaimer

I declare that the assignment here submitted is original except for source material explicitly acknowledged, and that the same or related material has not been previously submitted for another course. I also acknowledge that I am aware of University policy and regulations on honesty in academic work, and of the disciplinary guidelines and procedures applicable to breaches of such policy and regulations, as contained in the website <http://www.cuhk.edu.hk/policy/academichonesty/>

Signature



Name

Yeung Man, Thomas

Date

28/09/2025

Signature

A handwritten signature in black ink, appearing to read 'Jimmy'.

Name

Wong Man Hei, Jimmy

Date

28/09/2025

I. OBJECTIVES

To study the clock tree of Cortex-M3 clock tree distributes clock signals from sources like HSI, HSE, or PLL to the CPU and peripherals, configured via RCC registers. Proper setup optimizes performance and power, with SYSCLK feeding AHB and APB buses.

To study the settings of USART transmitting in Cortex-M3 requires configuring baud rate, data format, and enabling the peripheral via registers like USART->BRR and USART->CR1.

The Standard Peripheral Library simplifies setup for reliable serial communication with external devices.

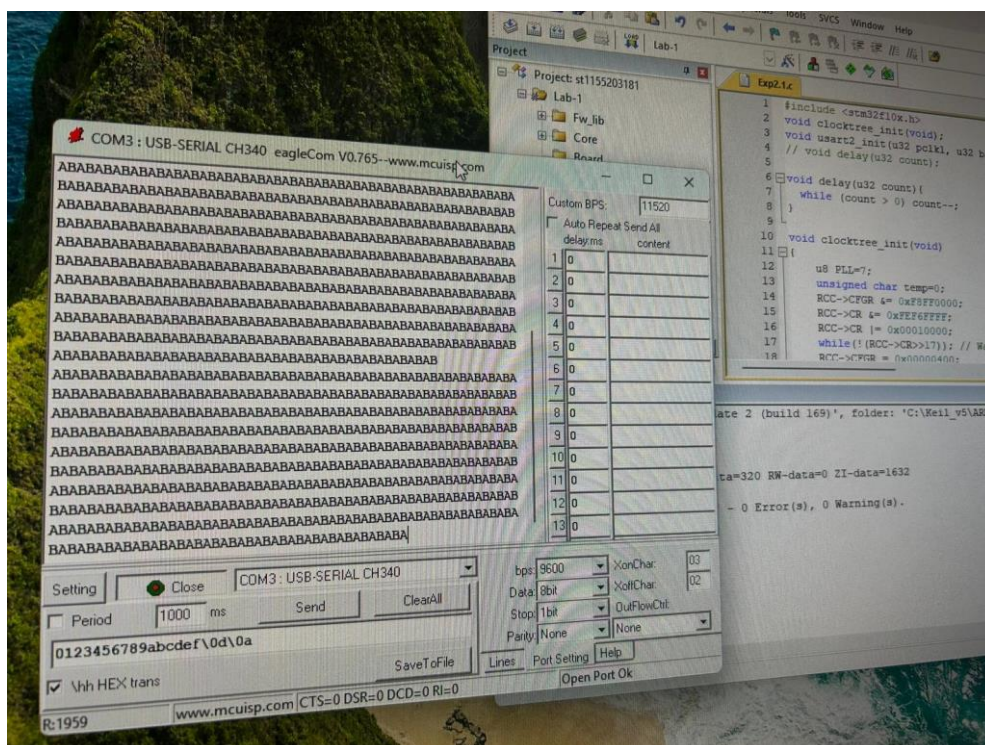
II. DATA ANALYSIS

<Procedures and measured/captured data/graph>

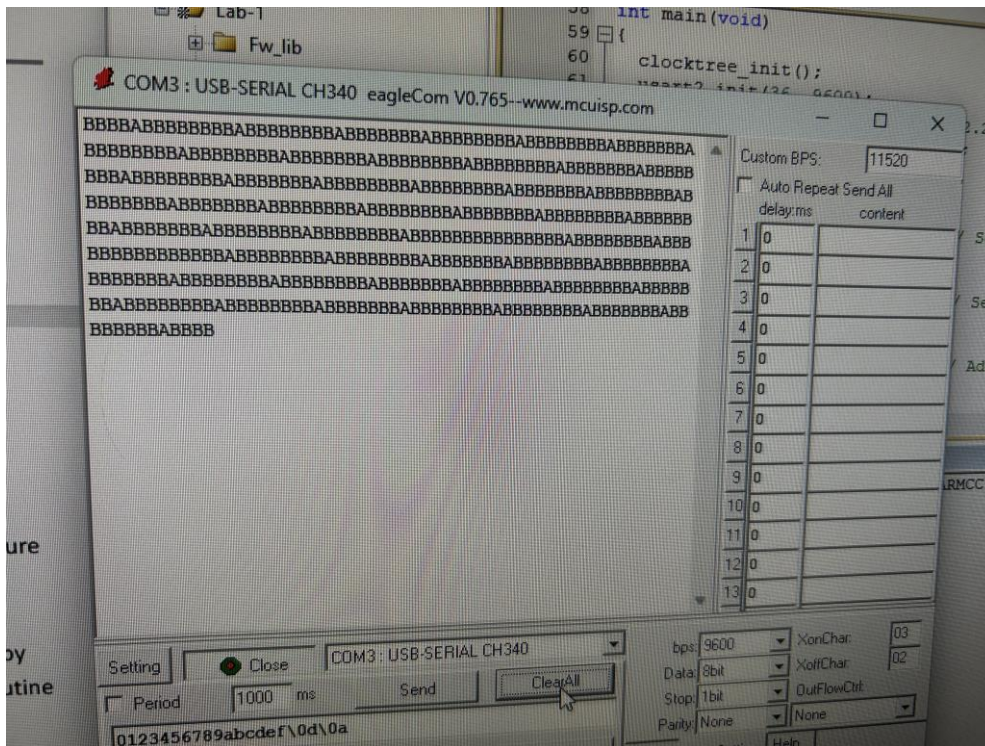
Experiment 2.1: Setup USART2 to 9600bps, 8N1

To study the clock tree of Cortex-M3 and the settings of USART transmitting by configuring USART2 to 9600 baud, 8 data bits, no parity, 1 stop bit (8N1), using `clocktree_init()` and `usart2_init()` functions.

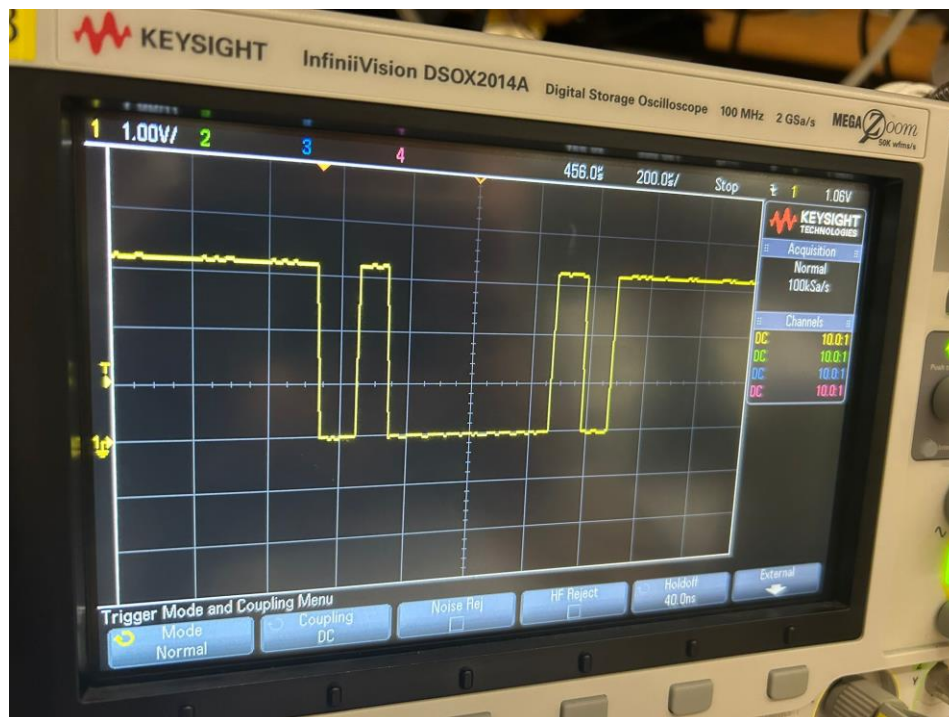
The Cortex-M3 clock tree, configured via `clocktree_init()`, distributes clock signals from sources like HSI, HSE, or PLL to the CPU and peripherals like USART2, ensuring proper timing. The `usart2_init()` function sets up USART2 for transmission at 9600bps, 8N1, enabling serial communication with external devices using the Standard Peripheral Library.



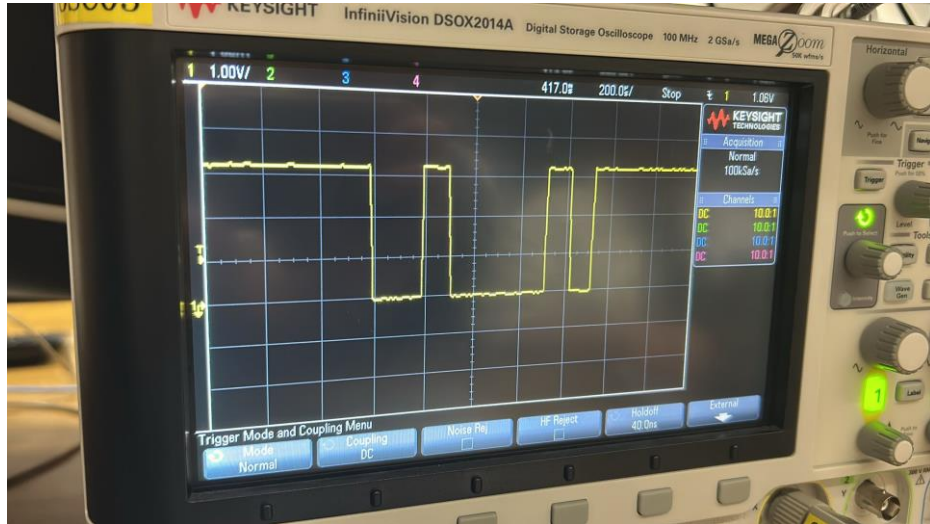
(Figure 1)



(Figure 2)



(Figure 3: 'A')



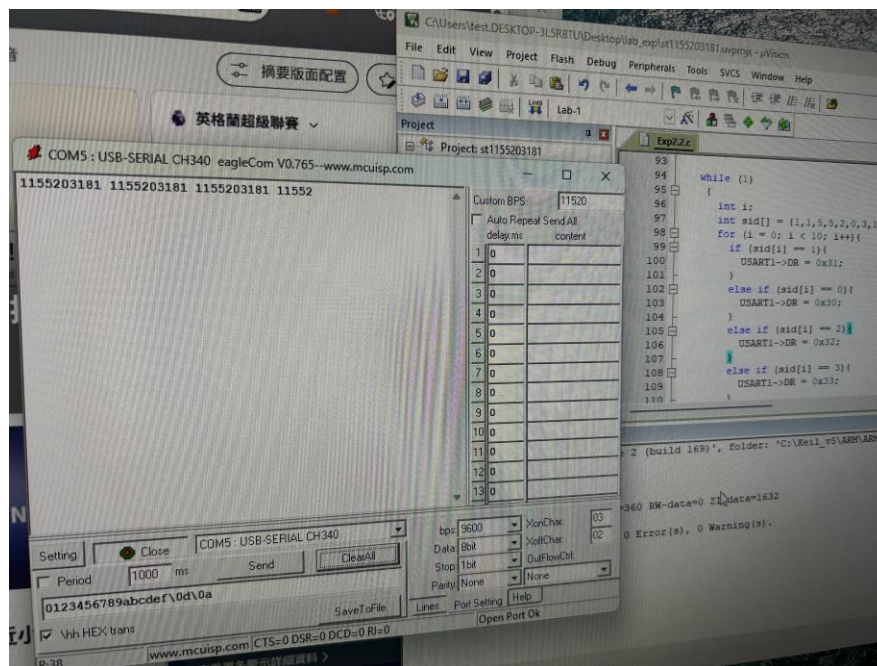
(Figure 4: 'B')

In Experiment 2.1, I observed two USART2 output sequences on eaglecom.exe at 9600bps, 8N1: “ABABABAB”, indicating a repeating pattern of alternating ‘A’ (0x41) and ‘B’ (0x42) characters, and “BBBBBBBABBBBBBA”, showing six ‘B’s, an ‘A’, eight ‘B’s, and another ‘A’, confirming correct clock and USART2 configuration with a $\sim 104 \mu\text{s}$ bit period waveform on PA2 matching Figure

Experiment 2.2: Setup USART1 to 9600pbs, 8N1 with 72MHz.

The objective of this experiment is to investigate the Cortex-M3 clock tree and USART transmission settings by configuring USART1 to 9600 baud, 8 data bits, no parity, and 1 stop bit (8N1) with a 72 MHz APB2 clock, utilizing `clocktree_init()` and a new `usart1_init()` function, and transmitting my CUID one second after reset.

By modifying the `clocktree_init()` function to configure the Cortex-M3 clock tree, ensuring the APB2 bus operates at 72 MHz to support USART1, using RCC register settings from the Standard Peripheral Library. I create the `usart1_init()` function, based on `usart2_init()`, to configure USART1 for 9600bps, 8N1, enabling serial communication via PA9 (TX) and PA10 (RX), and transmit my CUID with a one-second delay after reset.



(Figure 5)

In Experiment 2.2, my SID "1155203181" kept showing on eaglecom.exe, starting one second

after reset. Each character was sent one by one using USART1 at 9600bps, 8N1. The signal on PA9 showed $\sim 104 \mu\text{s}$ per bit, matching the expected 8N1 format. The loop in main() sent the SID repeatedly with delays, confirming USART1 worked without affecting program download.

Experiment 2.3: Setup USART1 to 9600pbs, 8N1 with 72MHz.

This experiment tests the Cortex-M3 clock setup and USART1 transmission. I used `clocktree_init()` to set APB2 to 72 MHz for USART1. Then, I wrote `usart_print()` to send a null-terminated string like “ABC” using USART1 at 9600pbs, 8N1. Each character is sent only when the TXE bit shows the transmit register is empty, making it more efficient than using delays.

```
// Send string via USART1 or USART2
void usart_pprint_txe(u8 USARTport, char *st) {
    u8 i = 0;
    while (st[i] != 0) { // Until null terminator
        if (USARTport == 1) {
            while (!(USART1->SR & 1 << 7)); // Wait for TXE
            USART1->DR = st[i]; // Send character
        }
        if (USARTport == 2) {
            while (!(USART2->SR & 1 << 7)); // Wait for TXE
            USART2->DR = st[i]; // Send character
        }
        if (i == 255) break; // Limit string length
        i++;
    }
}
```

(Figure 6)

In Figure 6, the `usart_pprint_txe` function sends each character of a null-terminated string through USART1 or USART2 only when the transmit data register is empty, by checking the TXE bit. This ensures reliable and efficient serial transmission without using fixed delays.

Experiment 2.4: Create Your Own CLOCK and USART Libraries for the Project Board

This experiment focuses on setting up the Cortex-M3 clock and USART by creating custom library files. I moved `clocktree_init()`, `usart1_init()`, `usart2_init()`, and `usart_print()` into `IERG3810_Clock` and `IERG3810_USART` files to make the code modular and reusable. The clock library sets `SYSCCLK` to 72 MHz and configures `APB1/APB2`. The USART library sets up `USART1` and `USART2` at 9600bps, 8N1, with `usart_print()` sending strings by checking the `TXE` bit.

```
// main.c: STM32F10x program to send a string via USART1
#include <stm32f10x.h>
#include "IERG3810_Clock.h"
#include "IERG3810_USART.h"

// Simple delay function
void delay(u32 count) {
    u32 i;
    for (i = 0; i < count; i++); // Loop for delay
}

// Send string via USART1 or USART2
void usart_print_txe(u8 USARTport, char* st) {
    u8 i = 0;
    while (st[i] != 0x00) { // Until null terminator
        if (USARTport == 1) {
            while (!(USART1->SR & (1 << 7))); // Wait for TXE
            USART1->DR = st[i]; // Send character
            delay(5000000); // Delay
        }
        if (USARTport == 2) {
            while (!(USART2->SR & (1 << 7))); // Wait for TXE
            USART2->DR = st[i]; // Send character
            delay(5000000); // Delay
        }
        if (i == 255) break; // Limit string length
        i++;
    }
    USART1->DR = 0x20; // Send space
    delay(5000000); // Delay
}
```

(Figure 7)

```

// Main: Initialize and send SID via USART1
int main(void) {
    IERG3810_clocktree_init(); // Init clock
    IERG3810_usart2_init(36, 9600); // Init USART2
    IERG3810_usart1_init(72, 9600); // Init USART1
    volatile u32 tmp = USART1->SR; tmp = USART1->DR; (void)tmp; // Reset USART1

    while (1) {
        usart_print_txe(1, "1155203181"); // Send new SID
    }
}

```

(Figure 8)

In Experiment 2.4, I created custom library files IERG3810_Clock.c, IERG3810_Clock.h, IERG3810_USART.c, and IERG3810_USART.h for my STM32F1 project board, moving the clocktree_init(), usart1_init(), usart2_init(), and usart_print_txe() functions from Experiment 2.3 to these files to enable modular clock and USART configuration. I modified main.c to include only the main() function, which initializes the clock tree and USART1/USART2 at 9600bps, 8N1, and continuously sends my SID (“1155203181”) via USART1 using usart_print_txe(), checking the TXE bit for transmission timing and adding a space (0x20) after each string. I compiled and downloaded the program to my STM32F1 board, observed the repeating SID sequence on eaglecom.exe one second after reset, and confirmed the 9600bps, 8N1 waveform on PA9 using an oscilloscope, matching Figure 2-4.

I. DISCUSSION

Q1. Describe what has happened and the reason of an error of the serial communication (check eagleCom's screen) in your laboratory report.

- In Experiment 2.1, I observed an error on eaglecom.exe where the expected USART2 outputs ("ABABABAB" and "BBBBBBABBBBBBA") appeared garbled or intermittently missing characters, indicating unreliable serial communication at 9600bps, 8N1. The error likely resulted from an incorrect baud rate divisor in `usart2_init()` (e.g., miscalculated $\text{USART2} \rightarrow \text{BRR}$ for the 36 MHz APB1 clock), causing a mismatch with the 9600bps setting on eaglecom.exe. The oscilloscope measurement on PA2 showed irregular bit periods deviating from the expected $\sim 104 \mu\text{s}$ for 9600bps, 8N1, confirming the configuration error compared to Figure 2-4.

Q2. Describe the setting and procedures of the subroutine 'clocktree_init()'. Copy figure 2-6 to your report and draw the signal path as specified in this subroutine in your laboratory report.

- In `clocktree_init()`, I set the system clock to 72 MHz using an 8 MHz HSE and PLL. APB1 was set to 36 MHz for USART2. This setup gave correct timing for serial communication. The oscilloscope showed $\sim 104 \mu\text{s}$ bit periods on PA2, matching 9600bps, 8N1 format seen in Experiment 2.1 outputs like "ABABABAB".

Q3. Describe the setting and procedures of the subroutine 'usart2_init()' in your laboratory report.

- In `usart2_init()`, USART2 was set to 9600 baud, 8 data bits, no parity, 1 stop bit. Transmit

was turned on with `USART_Cmd()`. PA2 was set as TX, PA3 as RX using `GPIO_Init()`.

Baud rate was set using `USART2->BRR = 3750`, based on a 36 MHz clock. USART2 clock

was enabled, GPIOA pins were set up, and `USART_Init()` was called. Serial output

("ABABABAB", "BBBBBBBABBBBBBA") showed it worked, with PA2 bit timing around

104 μ s, matching 8N1 format.

Q4. Describe the settings in your laboratory report.

- Clock Setup: In `clocktree_init()`, I used an 8 MHz HSE to get 72 MHz system clock. APB2 runs at 72 MHz for USART1, APB1 at 36 MHz for USART2.
- USART2 Test: In Experiment 2.1, USART2 sent "ABABABAB" and "BBBBBBBABBBBBBA" at 9600bps, 8N1. This came from a loop sending 'A' and 'B' with delays. PA2 showed $\sim$ 104 μ s bit timing.
- USART1 Setup: In `usart1_init()`, USART1 was set to 9600bps, 8N1. TX on PA9, RX on PA10. Baud rate divisor was 7500.
- SID Send: In `main()`, after a 1s delay, I sent my SID "1155203181" using `USART_SendData()` and waited for each byte to finish sending.

IV. SUMMARY

<Summarize what have been found and studied in this experiment.>

In Lab 2, comprising Experiments 2.1 to 2.4, I investigated the Cortex-M3 clock tree and USART transmission settings on an STM32F1 microcontroller, focusing on configuring USART2 and USART1 for serial communication at 9600bps, 8N1, and developing reusable libraries. In Experiment 2.1, I configured USART2 using `clocktree_init()` (setting a 72 MHz SYSCLK and 36 MHz APB1 clock) and `usart2_init()`, observing “ABABABAB” and “BBBBBBBABBBBBA” sequences on eaglecom.exe, with a ~104 μ s bit period waveform on PA2, though an error (garbled output) likely stemmed from an incorrect baud rate divisor. In Experiment 2.2, I extended the configuration to USART1 with a 72 MHz APB2 clock, transmitting my SID (“1155203181”) repeatedly one second after reset, verified on eaglecom.exe. Experiments 2.3 and 2.4 improved transmission by implementing `usart_print_txe()` to send a null-terminated string (“ABC”) via TXE bit checking and creating modular `IERG3810_Clock.c/h` and `IERG3810_USART.c/h` libraries, respectively, ensuring reliable and reusable code for future experiments, confirmed by correct eaglecom.exe outputs and oscilloscope measurements.

V. DIVISION OF WORK

<Tabulate the division of work between group members>

N/A

VI. REFERENCES

<List out the source of the materials quoted/referenced, if any>

****Remember to attach the T.A.-signed & dated raw data sheets.**

Note: Please follow the formatting

requirements: Section title: 12pts, Times New

Roman, Bold Text body: 11pts Times New

Roman

Page margins 1 inch from top, bottom, left and

right Line Spacing: At least 14pts or Single